

Logging and Periodic Auditing

1. Overview

Not all failures are obvious.

Some systems return **success responses**, but the actual write may still be:

- Missing
- Corrupted
- Partially applied
- Delayed
- Written with wrong values
- Overwritten later
- Lost due to replication issues

These are called **silent errors** because no immediate exception is raised.

To detect such issues, systems use:

Logging + Periodic Auditing

1. Log every write request
2. Later run audit jobs
3. Compare expected data vs actual data
4. Detect mismatches
5. Repair inconsistencies

2. Why This Is Needed

Suppose:

None

Order Service → Inventory DB

API returns:

None

200 OK

But actual inventory update failed internally.

Your application believes success happened, but database state is wrong.

Without auditing:

✗ Error may remain hidden for days

With auditing:

✓ Background job detects mismatch

3. What Are Silent Errors?

Silent errors are failures not visible in normal request flow.

Examples:

- Network acknowledged request but write not committed
- DB replication lag loses update
- Bug writes wrong field
- Duplicate overwrite
- Third-party API says success but drops request
- Data truncation
- Serialization issue
- Partial transaction

4. Core Idea

Step 1: Log Every Important Write

Whenever system performs write:

None

Update wallet balance +100

Change order status = shipped

Update user email

Reduce stock = 2

Write structured logs.

Step 2: Run Periodic Audit Job

Audit process checks:

None

Expected state from logs

vs

Actual state in target system

If mismatch:

- Alert team
- Auto-repair
- Replay write
- Investigate bug

5. Example: Payment System

User pays ₹500.

System logs:

None

`txn_id=TX123`

`wallet_credit=500`

`status=success`

Later audit checks:

None

`Wallet balance updated?`

`Transaction exists?`

`Ledger matches?`

If not:

None

`Mismatch detected`

Critical issue found.

6. Example: E-commerce Order State

Expected:

None

Order #9001 = SHIPPED

Actual DB:

None

Order #9001 = PAID

Audit catches stuck order.

7. What to Log

Use structured logs with identifiers.

Store:

None

timestamp

request_id

user_id

entity_id

operation type

old value

new value

service name

status

trace_id

response code

Example:

JSON

```
{  
  "order_id": 9001,  
  "action": "update_status",  
  "new_status": "SHIPPED",  
  "time": "10:05"  
}
```


8. Where Logs Are Stored

Production systems use centralized logging:

- Elasticsearch / OpenSearch
- Splunk
- Datadog
- CloudWatch
- Kafka log pipeline
- S3 archival storage

Need durable searchable logs.

9. Periodic Auditing Types

A. Full Audit

Check all records.

None

Compare all user balances nightly

Pros:

- Complete

Cons:

- Expensive

B. Incremental Audit

Check recent changes only.

None

Last 1 hour updates

Faster and common.

C. Sample Audit

Check random subset.

Useful for large systems.

10. Reconciliation Pattern

Often called **reconciliation** in fintech.

Compare:

None

Internal ledger

vs

Bank statement

vs

Payment gateway records

If mismatch → resolve.

Used heavily in:

- Stripe-like systems
- Banking
- Accounting

11. Audit Flow

None

Write Request Happens

↓

Create Durable Log Entry

↓

Later Audit Job Runs

↓

Fetch Expected State

↓

Compare With Actual State

↓

Mismatch?

Yes → Alert / Repair

No → Mark Clean

12. Auto Repair Strategies

If mismatch found:

Replay Missing Write

None

Inventory decrement missing

→ replay event

Correct Value

None

Wrong balance

→ recompute from ledger

Manual Review

Used for critical systems.

13. Real-World Use Cases

Payment Systems

- Transaction reconciliation
- Missing credits
- Duplicate charges

Inventory Systems

- Stock mismatch

Shipping

- Delivered vs status not updated

User Profiles

- Email change not persisted

Analytics

- Event count mismatch

14. Why Logs Help Fault Tolerance

Logs create an **audit trail**.

Even if systems fail later:

- Know what should have happened
- Can replay operations
- Diagnose bugs
- Recover lost writes

15. Important Design Considerations

A. Idempotency

Replayed writes should not duplicate effects.

B. Immutable Logs

Avoid changing historical logs.

C. Time Ordering

Need timestamps / sequence IDs.

D. Retention

Keep logs long enough.

E. Sensitive Data

Mask PII / secrets.

16. Interview Answer

Q: How do you detect silent data inconsistencies in distributed systems?

Answer:

I'd log all important write operations in durable structured logs, then run periodic audit/reconciliation jobs that compare expected state with actual state in downstream systems. If mismatches are found, we alert or automatically repair via replay.

17. Advanced Techniques

Event Sourcing

Logs themselves become source of truth.

Checksums

Detect corruption.

Dual Writes Verification

If writing to two systems, audit both.

Real-Time Auditing

Continuous stream-based reconciliation.

18. Best Practices

- Use structured logs, not plain text
- Include trace IDs
- Make logs durable
- Run regular audits
- Alert on mismatches
- Build replay tools
- Encrypt sensitive logs
- Keep retention policy

19. Short Revision Notes

None

Silent Error = write appears successful but data wrong/missing

Solution:

1. Log all writes
2. Periodic audit job
3. Compare expected vs actual
4. Repair mismatch

Used in:

Payments, inventory, finance, orders